Problem A. Scrambled Scrabble

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

You are playing a word game using a standard set of 26 uppercase English letters: `A` — `Z`. In this game, you can form vowels and consonants as follows.

- The letters `A`, `E`, `I`, `O`, and `U` can only form a vowel.
- The letter `Y` can form either a vowel or a consonant.
- Each of the remaining letters other than `A`, `E`, `I`, `O`, `U`, and `Y` can only form a consonant.
- The string `NG` can form a single consonant when concatenated together.

Denote a syllable as a concatenation of a consonant, a vowel, and a consonant in that order. A word is a concatenation of one or more syllables.

You are given a string S and you want to create a word from it. You are allowed to delete zero or more letters from S and rearrange the remaining letters to form the word. Find the length of the longest word that can be created, or determine if no words can be created.

Input

A single line consisting of a string S ($1 \leq |S| \leq 5000$). The string S consists of only uppercase English letters.

Output

If a word cannot be created, output `0`. Otherwise, output a single integer representing the length of longest word that can be created.

Examples

standard input	standard output
ICPCJAKARTA	9
NGENG	5
YYY	3
DANGAN	6
AEIOUY	0

Note

Explanation for the sample input/output #1

A possible longest word is `JAKCARTAP`, consisting of the syllables `JAK`, `CAR`, and `TAP`.

Explanation for the sample input/output #2

The whole string S is a word consisting of one syllable which is the concatenation of the consonant `NG`, the vowel `E`, and the consonant `NG`.

Explanation for the sample input/output #3

The whole string S is a word consisting of one syllable which is the concatenation of the consonant `Y`, the vowel `Y`, and the consonant `Y`.

Explanation for the sample input/output #4

The whole string S is a word consisting of two syllables: `DAN` and `GAN`.

Problem B. ICPC Square

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

ICPC Square is a hotel provided by the ICPC Committee for the accommodation of the participants. It consists of N floors (numbered from 1 to N). This hotel has a very unique elevator. If a person is currently at floor x, by riding the elevator once, they can go to floor y if and only if y is a multiple of x and $y - x \leq D$.

You are currently at floor S. You want to go to the highest possible floor by riding the elevator zero or more times. Determine the highest floor you can reach.

Input

A single line consisting of three integers N D S ($2 \leq N \leq 10^{12}; 1 \leq D \leq N - 1; 1 \leq S \leq N$).

Output

Output a single integer representing the highest floor you can reach by riding the elevator zero or more times.

Examples

standard input	standard output
64 35 3	60
2024 2023 1273	1273

Note

Explanation for the sample input/output #1

First, ride the elevator from floor 3 to floor 15. This is possible because 15 is a multiple of 3 and $15-3 \leq 35$. Then, ride the elevator from floor 15 to floor 30. This is possible because 30 is a multiple of 15 and $30 - 15 \leq 35$. Finally, ride the elevator from floor 30 to floor 60. This is possible because 60 is a multiple of 30 and $60 - 30 \leq 35$.

Problem C. Saraga

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

The word saraga is an abbreviation of sarana olahraga, an Indonesian term for a sports facility. It is created by taking the prefix `sara` of the word `sarana` and the suffix `ga` of the word `olahraga`. Interestingly, it can also be created by the prefix `sa` of the word `sarana` and the suffix `raga` of the word `olahraga`.

An abbreviation of two strings S and T is interesting if there are at least two different ways to split the abbreviation into two **non-empty** substrings such that the first substring is a prefix of S and the second substring is a suffix of T.

You are given two strings S and T. You want to create an interesting abbreviation of strings S and T with minimum length, or determine if it is impossible to create an interesting abbreviation.

Input

The first line consists of a string S ($1 \leq |S| \leq 200\,000$).

The second line consists of a string T ($1 \leq |T| \leq 200\,000$).

Both strings S and T only consist of lowercase English letters.

Output

If it is impossible to create an interesting abbreviation, output `-1`.

Otherwise, output a string in a single line representing an interesting abbreviation of strings S and T with minimum length. If there are multiple solutions, output any of them.

Examples

standard input	standard output
sarana olahraga	saga
berhiber wortelhijau	berhijau
icpc icpc	icpc
icpc jakarta	-1

Note

Explanation for the sample input/output #1

You can split `saga` into `s` and `aga`, or `sa` and `ga`. The abbreviation `saraga` is interesting, but `saga` has a smaller length.

Explanation for the sample input/output #2

The abbreviation `belhijau` is also interesting with minimum length, so it is another valid solution.

Problem D. Aquatic Dragon

Input file:	standard input
Output file:	standard output
Time limit:	3 seconds
Memory limit:	1024 megabytes

You live in an archipelago consisting of N islands (numbered from 1 to N) laid out in a single line. Island i is adjacent to island $i+1$, for $1 \leq i < N$. Between adjacent islands i and $i+1$, there is a pair of one-directional underwater tunnels: one that allows you to walk from island i to island $i+1$ and one for the opposite direction. Each tunnel can only be traversed **at most once**.

You also have a dragon with you. It has a stamina represented by a non-negative integer. The stamina is required for the dragon to perform its abilities: swim and fly. Initially, its stamina is 0.

Your dragon's stamina can be increased as follows. There is a magical shrine on each island that you can activate if you are on that island. The moment you activate the shrine on island i, your dragon's stamina is increased by P_i regardless of the dragon's current location. Each shrine can be activated **at most once** and the action of activating a shrine takes no time.

When you are on an island, there are 3 moves that you can perform.

- **Swim with your dragon** to an adjacent island if your dragon and you are on the same island. You can perform if your dragon's stamina is at least D. This move reduces your dragon's stamina by D, and it takes T_s seconds to perform.
- **Fly with your dragon** to an adjacent island if your dragon and you are on the same island. You can perform this move if your dragon's stamina is not 0. This move sets your dragon's stamina to 0, and it takes T_f seconds to perform.
- **Walk alone without your dragon** to an adjacent island through the underwater tunnel. This move takes T_w seconds to perform. Once you walk through this tunnel, it cannot be used again.

Note that both swimming and flying do not use tunnels.

Your dragon and you are currently on island 1. Your mission is to go to island N **with your dragon**. Determine the minimum possible time to complete your mission.

Input

The first line consists of five integers N D T_s T_f T_w ($2 \leq N \leq 200\,000; 1 \leq D, T_s, T_f, T_w \leq 200\,000$).

The second line consists of N integers P_i ($1 \leq P_i \leq 200\,000$).

Output

Output an integer in a single line representing the minimum possible time to go to island N **with your dragon**.

Examples

standard input	standard output
5 4 2 9 1 1 2 4 2 1	28
5 4 2 1 1 1 2 4 2 1	4
3 4 2 10 1 3 1 2	16

Note

Explanation for the sample input/output #1

The following sequence of moves will complete your mission in the minimum time.

1. Activate the shrine on island 1. Your dragon's stamina is now 1.
2. Fly with your dragon to island 2, then activate the shrine on island 2. Your dragon's stamina is now 2.
3. Walk alone to island 3, then activate the shrine on island 3. Your dragon's stamina is now 6.
4. Walk alone to island 4, then activate the shrine on island 4. Your dragon's stamina is now 8.
5. Walk alone to island 3.
6. Walk alone to island 2.
7. Swim with your dragon to island 3. Your dragon's stamina is now 4.
8. Swim with your dragon to island 4. Your dragon's stamina is now 0.
9. Walk alone to island 5, then activate the shrine on island 5. Your dragon's stamina is now 1.
10. Walk alone to island 4.
11. Fly with your dragon to island 5.

Explanation for the sample input/output #2

Repeat the following process for $1 \leq i < 5$: Activate the shrine on island i, then fly with your dragon to island $i + 1$.

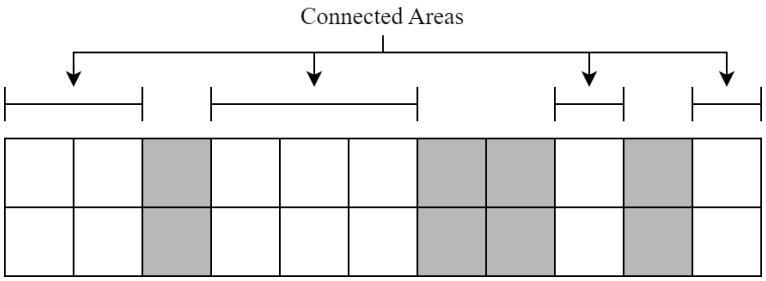
Problem E. Narrower Passageway

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

You are a strategist of The ICPC Kingdom. You received an intel that there will be monster attacks on a narrow passageway near the kingdom. The narrow passageway can be represented as a grid with 2 rows (numbered from 1 to 2) and N columns (numbered from 1 to N). Denote (r, c) as the cell in row r and column c. A soldier with a power of $P_{r,c}$ is assigned to protect (r, c) every single day.

It is known that the passageway is very foggy. Within a day, each column in the passageway has a 50% chance of being covered in fog. If a column is covered in fog, the two soldiers assigned to that column are not deployed that day. Otherwise, the assigned soldiers will be deployed.

Define a connected area $[u, v]$ $(u \le v)$ as a maximal set of consecutive columns from u to v (inclusive) such that each column in the set is not covered in fog. The following illustration is an example of connected areas. The grayed cells are cells covered in fog. There are 4 connected areas: $[1, 2]$, $[4, 6]$, $[9, 9]$, and $[11, 11]$.



The strength of a connected area $[u, v]$ can be calculated as follows. Let m_1 and m_2 be the maximum power of the soldiers in the first and second rows of the connected area, respectively. Formally, $m_r = \max(P_{r,u}, P_{r,u+1}, \ldots, P_{r,v})$ for $r \in \{1, 2\}$. If $m_1 = m_2$, then the strength is 0. Otherwise, the strength is $\min(m_1, m_2)$.

The total strength of the deployment is the sum of the strengths for all connected areas. Determine the expected total strength of the deployment on any single day.

Input

The first line consists of an integer N $(1 \le N \le 100\,000)$.

Each of the next two lines consists of N integers $P_{r,c}$ $(1 \le P_{r,c} \le 200\,000)$.

Output

Let $M = 998\,244\,353$. It can be shown that the expected total strength can be expressed as an irreducible fraction $\frac{x}{y}$ such that x and y are integers and $y \not\equiv 0 \pmod{M}$. Output an integer k in a single line such that $0 \le k < M$ and $k \cdot y \equiv x \pmod{M}$.

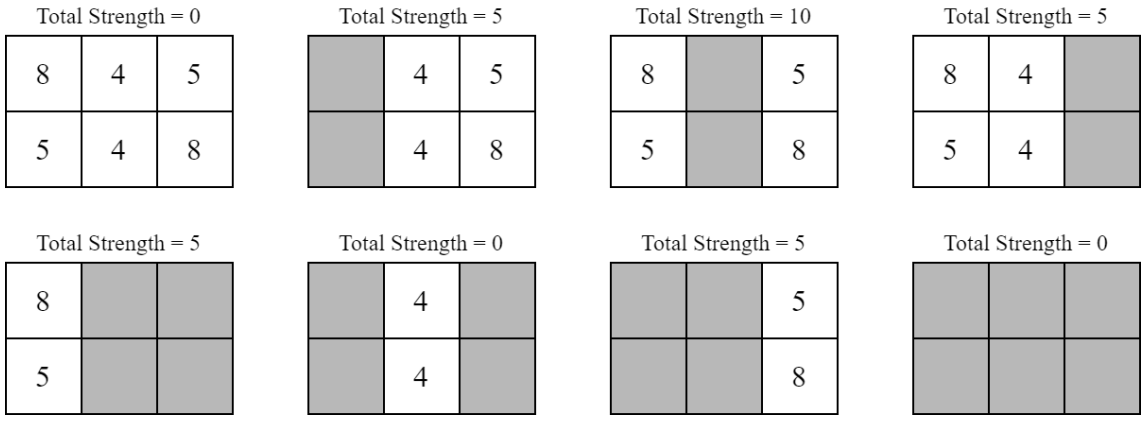
Examples

standard input	standard output
3 8 4 5 5 4 8	249561092
5 10 20 5 8 5 5 20 7 5 8	811073541

Note

Explanation for the sample input/output #1

There are 8 possible scenarios for the passageway.



Each scenario is equally likely to happen. Therefore, the expected total strength is $(0+5+10+5+5+0+5+0)/8 = \frac{15}{4}$. Since $249\,561\,092 \cdot 4 \equiv 15 \pmod{998\,244\,353}$, the output of this sample is 249 561 092.

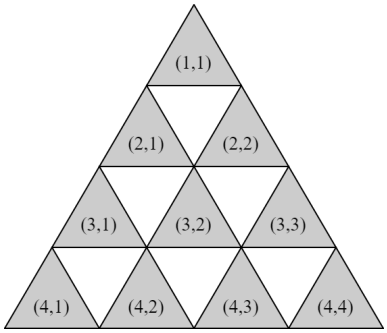
Explanation for the sample input/output #2

The expected total strength is $\frac{67}{16}$.

Problem F. Grid Game 3-angle

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

Your friends, Anda and Kamu decide to play a game called **Grid Game** and ask you to become the gamemaster. As the gamemaster, you set up a triangular grid of size N. The grid has N rows (numbered from 1 to N). Row r has r cells; the c-th cell of row r is denoted as (r, c).

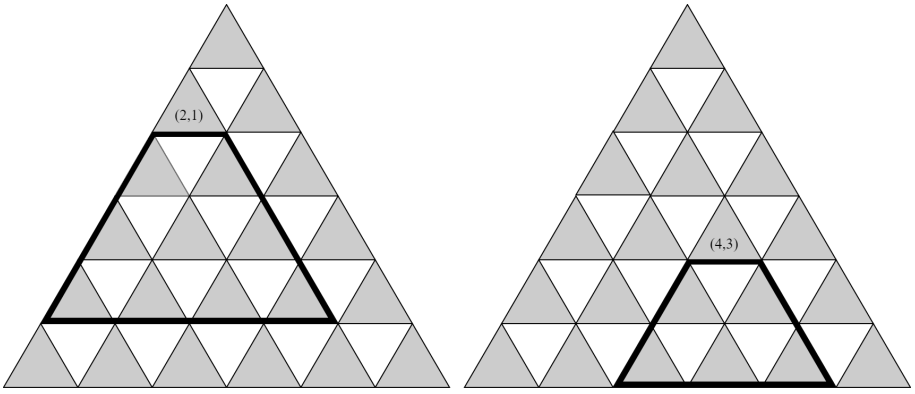


Before the game starts, M different cells (numbered from 1 to M) are chosen: at cell (R_i, C_i), you add A_i stones on it. You then give Anda and Kamu an integer K and commence the game.

Anda and Kamu will take turns alternately with Anda taking the first turn. A player on their turn will do the following.

- Choose a cell (r, c) with at least one stone on it.
- Remove **at least one** but **at most** K stones from the chosen cell.
- For each cell (x, y) such that $r + 1 \le x \le \min(N, r + K)$ and $c \le y \le c + x - r$, add **zero or more** stones but **at most** K stones to cell (x, y).

The following illustrations show all the possible cells in which you can add stones for $K = 3$. You choose the cell $(2, 1)$ for the left illustration and the cell $(4, 3)$ for the right illustration.



A player who is unable to complete their turn (because there are no more stones on the grid) will lose the game, and the opposing player wins. Determine who will win the game if both players play optimally.

Input

This problem is a multi-case problem. The first line consists of an integer T ($1 \le T \le 100$) that represents the number of test cases.

Each test case starts with a single line consisting of three integers N M K ($1 \leq N \leq 10^9; 1 \leq M, K \leq 200\,000$). Then, each of the next M lines consists of three integers R_i C_i A_i ($1 \leq C_i \leq R_i \leq N; 1 \leq A_1 \leq 10^9$). The pairs (R_i, C_i) are distinct.

The sum of M across all test cases does not exceed 200 000.

Output

For each case, output a string in a single line representing the player who will win the game if both players play optimally. Output `Anda` if Anda, the first player, wins. Otherwise, output `Kamu`.

Example

standard input	standard output
3 2 2 4 1 1 3 2 1 2 100 2 1 4 1 10 4 4 10 10 5 2 1 1 4 3 1 2 4 2 5 2 2 1 5 3 4	Anda Kamu Anda

Note

Explanation for the sample input/output #1

For the first case, during the first turn, Anda will remove all the stones from cell $(1, 1)$ and then add three stones at $(2, 1)$. The only cell with stones left is now cell $(2, 1)$ with five stones, so Kamu must remove stones from that cell. No matter how many stones are removed by Kamu, Anda can remove all the remaining stones at $(2, 1)$ and win the game.

For the second case, Kamu can always mirror whatever move made by Anda until Anda can no longer complete their turn.

Problem G. X Aura

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

Mount ICPC can be represented as a grid of R rows (numbered from 1 to R) and C columns (numbered from 1 to C). The cell located at row r and column c is denoted as (r, c) and has a height of $H_{r,c}$. Two cells are adjacent to each other if they share a side. Formally, (r, c) is adjacent to $(r-1, c)$, $(r+1, c)$, $(r, c-1)$, and $(r, c+1)$, if any exists.

You can move only between adjacent cells, and each move comes with a penalty. With an aura of an **odd positive integer** X, moving from a cell with height h_1 to a cell with height h_2 gives you a penalty of $(h_1 - h_2)^X$. Note that the penalty can be negative.

You want to answer Q independent scenarios. In each scenario, you start at the starting cell (R_s, C_s) and you want to go to the destination cell (R_f, C_f) with minimum total penalty. In some scenarios, the total penalty might become arbitrarily small; such a scenario is called invalid. Find the minimum total penalty to move from the starting cell to the destination cell, or determine if the scenario is invalid.

Input

The first line consists of three integers R C X ($1 \le R, C \le 1000; 1 \le X \le 9; X$ is an odd integer).

Each of the next R lines consists of a string H_r of length C. Each character in H_r is a number from `0` to `9`. The c-th character of H_r represents the height of cell (r, c), or $H_{r,c}$.

The next line consists of an integer Q ($1 \le Q \le 100\,000$).

Each of the next Q lines consists of four integers R_s C_s R_f C_f ($1 \le R_s, R_f \le R; 1 \le C_s, C_f \le C$).

Output

For each scenario, output the following in a single line. If the scenario is invalid, output `INVALID`. Otherwise, output a single integer representing the minimum total penalty to move from the starting cell to the destination cell.

Examples

standard input	standard output
3 4 1 3359 4294 3681 5 1 1 3 4 3 3 2 1 2 2 1 4 1 3 3 2 1 1 1 1	2 4 -7 -1 0
2 4 5 1908 2023 2 1 1 2 4 1 1 1 1	INVALID INVALID
3 3 9 135 357 579 2 3 3 1 1 2 2 2 2	2048 0

Note

Explanation for the sample input/output #1

For the first scenario, one of the solutions is to move as follows: $(1,1) \to (2,1) \to (3,1) \to (3,2) \to (3,3) \to (3,4)$. The total penalty of this solution is $(3-4)^1 + (4-3)^1 + (3-6)^1 + (6-8)^1 + (8-1)^1 = 2$.

Explanation for the sample input/output #2

For the first scenario, the cycle $(1,1) \to (2,1) \to (2,2) \to (1,2) \to (1,1)$ has a penalty of $(1-2)^5 + (2-0)^5 + (0-9)^5 + (9-1)^5 = -26250$. You can keep repeating this cycle to make your total penalty arbitrarily small. Similarly, for the second scenario, you can move to $(1,1)$ first, then repeat the same cycle.

Problem H. Missing Separators

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

You have a dictionary, which is a list of **distinct** words sorted in alphabetical order. Each word consists of uppercase English letters.

You want to print this dictionary. However, there is a bug with the printing system, and all words in the list are printed next to each other without any separators between words. Now, you ended up with a string S that is a concatenation of all the words in the dictionary in the listed order.

Your task is to reconstruct the dictionary by splitting S into one or more words. Note that the reconstructed dictionary must consist of distinct words sorted in alphabetical order. Furthermore, you want to maximize the number of words in the dictionary. If there are several possible dictionaries with the maximum number of words, you can choose any of them.

Input

A single line consisting of a string S ($1 \leq |S| \leq 5000$). String S consists of only uppercase English letters.

Output

First, output an integer in a single line representing the maximum number of the words in the reconstructed dictionary. Denote this number as n.

Then, output n lines, each containing a single string representing the word. The words must be distinct, and the list must be sorted alphabetically. The concatenation of the words in the listed order must equal S.

If there are several possible dictionaries with the maximum number of words, output any of them.

Examples

standard input	standard output
ABACUS	4 A BA C US
AAAAAA	3 A AA AAA
EDCBA	1 EDCBA

Problem I. Microwavable Subsequence

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

You are given an array of N integers: $[A_1, A_2, \ldots, A_N]$.

A subsequence can be derived from an array by removing zero or more elements without changing the order of the remaining elements. For example, $[2, 1, 2]$, $[3, 3]$, $[1]$, and $[3, 2, 1, 3, 2]$ are subsequences of array $[3, 2, 1, 3, 2]$, while $[1, 2, 3]$ is not a subsequence of array $[3, 2, 1, 3, 2]$.

A subsequence is microwavable if the subsequence consists of **at most** two distinct values and each element differs from its adjacent elements. For example, $[2, 1, 2]$, $[3, 2, 3, 2]$, and $[1]$ are microwavable, while $[3, 3]$ and $[3, 2, 1, 3, 2]$ are not microwavable.

Denote a function $f(x, y)$ as the length of the longest microwavable subsequence of array A such that each element within the subsequence is either x or y. Find the sum of $f(x, y)$ for all $1 \leq x < y \leq M$.

Input

The first line consists of two integers N M ($1 \leq N, M \leq 300\,000$).

The second line consists of N integers A_i ($1 \leq A_i \leq M$).

Output

Output a single integer representing the sum of $f(x, y)$ for all $1 \leq x < y \leq M$.

Examples

standard input	standard output
5 4 3 2 1 3 2	13
3 3 1 1 1	2

Note

Explanation for the sample input/output #1

The value of $f(1, 2)$ is 3, taken from the subsequence $[2, 1, 2]$ that can be obtained by removing A_1 and A_4. The value of $f(1, 3)$ is 3, taken from the subsequence $[3, 1, 3]$ that can be obtained by removing A_2 and A_5. The value of $f(2, 3)$ is 4, taken from the subsequence $[3, 2, 3, 2]$ that can be obtained by removing A_3. The value of $f(1, 4)$, $f(2, 4)$, and $f(3, 4)$ are all 1.

Explanation for the sample input/output #2

The value of $f(1, 2)$ and $f(1, 3)$ are both 1, while the value of $f(2, 3)$ is 0.

Problem J. Xorderable Array

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

You are given an array A of N integers: $[A_1, A_2, \ldots, A_N]$.

The array A is (p, q)-xorderable if it is possible to rearrange A such that for each pair (i, j) that satisfies $1 \leq i < j \leq N$, the following conditions must be satisfied after the rearrangement: $A_i \oplus p \leq A_j \oplus q$ **and** $A_i \oplus q \leq A_j \oplus p$. The operator $\oplus$ represents the bitwise xor.

You are given another array X of length M: $[X_1, X_2, \ldots, X_M]$. Calculate the number of pairs (u, v) where array A is (X_u, X_v)-xorderable for $1 \leq u < v \leq M$.

Input

The first line consists of two integers N M ($2 \leq N, M \leq 200\,000$).

The second line consists of N integers A_i ($0 \leq A_i < 2^{30}$).

The third line consists of M integers X_u ($0 \leq X_u < 2^{30}$).

Output

Output a single integer representing the number of pairs (u, v) where array A is (X_u, X_v)-xorderable for $1 \leq u < v \leq M$.

Examples

standard input	standard output
3 4 0 3 0 1 2 1 1	3
5 2 0 7 13 22 24 12 10	1
3 3 0 0 0 1 2 3	0

Note

Explanation for the sample input/output #1

The array A is $(1, 1)$-xorderable by rearranging the array A to $[0, 0, 3]$.

Explanation for the sample input/output #2

The array A is $(12, 10)$-xorderable by rearranging the array A to $[13, 0, 7, 24, 22]$.

Problem K. GCDDCG

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

You are playing the Greatest Common Divisor Deck-Building Card Game (GCDDCG). There are N cards (numbered from 1 to N). Card i has the value of A_i, which is an integer between 1 and N (inclusive).

The game consists of N rounds (numbered from 1 to N). Within each round, you need to build two **non-empty** decks, deck 1 and deck 2. A card cannot be inside both decks, and it is allowed to not use all N cards. In round i, the greatest common divisor (GCD) of the card values in each deck must equal i.

Your creativity point during round i is the product of i and the number of ways to build two valid decks. Two ways are considered different if one of the decks contains different cards.

Find the sum of creativity points across all N rounds. Since the sum can be very large, calculate the sum modulo 998 244 353.

Input

The first line consists of an integer N ($2 \le N \le 200\,000$).

The second line consists of N integers A_i ($1 \le A_i \le N$).

Output

Output a single integer representing the sum of creativity points across all N rounds modulo 998 244 353.

Examples

standard input	standard output
3 3 3 3	36
4 2 2 4 4	44
9 4 2 6 9 7 7 7 3 3	10858

Note

Explanation for the sample input/output #1

The creativity point during each of rounds 1 and 2 is 0.

During round 3, there are 12 ways to build both decks. Denote B and C as the set of card numbers within deck 1 and deck 2, respectively. The 12 ways to build both decks are:

- $B = \{1\}, C = \{2\}$;
- $B = \{1\}, C = \{3\}$;
- $B = \{1\}, C = \{2, 3\}$;
- $B = \{2\}, C = \{1\}$;
- $B = \{2\}, C = \{3\}$;
- $B = \{2\}, C = \{1, 3\}$;
- $B = \{3\}, C = \{1\}$;

- $B = \{3\}, C = \{2\}$;
- $B = \{3\}, C = \{1, 2\}$;
- $B = \{1, 2\}, C = \{3\}$;
- $B = \{2, 3\}, C = \{1\}$; and
- $B = \{1, 3\}, C = \{2\}$.

Explanation for the sample input/output #2

For rounds 1, 2, 3 and 4, there are 0, 18, 0, and 2 ways to build both decks, respectively.

Problem L. Buggy DFS

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

You are currently studying a graph traversal algorithm called the Depth First Search (DFS). However, due to a bug, your algorithm is slightly different from the standard DFS. The following is an algorithm for your Buggy DFS (BDFS), assuming the graph has N nodes (numbered from 1 to N).

```
BDFS():
  let S be an empty stack
  let FLAG be a boolean array of size N which are all false initially
  let counter be an integer initialized with 0

  push 1 to S

  while S is not empty:
    pop the top element of S into u
    FLAG[u] = true

    for each v neighbour of u in ascending order:
      counter = counter + 1
      if FLAG[v] is false:
        push v to S

  return counter
```

You realized that the bug made the algorithm slower than standard DFS, which can be investigated by the return value of the function `BDFS()`. To investigate the behavior of this algorithm, you want to make some test cases by constructing an undirected simple graph such that the function `BDFS()` returns K, or determine if it is impossible to do so.

Input

A single line consisting of an integer K ($1 \leq K \leq 10^9$).

Output

If it is impossible to construct an undirected simple graph such that the function `BDFS()` returns K, then output `-1 -1` in a single line.

Otherwise, output the graph in the following format. The first line consists of two integers N and M, representing the number of nodes and undirected edges in the graph, respectively. Each of the next M lines consists of two integers u and v, representing an undirected edge that connects node u and node v. You are allowed to output the edges in any order. This graph has to satisfy the following constraints:

- $1 \leq N \leq 32\,768$
- $1 \leq M \leq 65\,536$
- $1 \leq u, v \leq N$, for all edges.
- The graph is a simple graph, i.e. there are no multi-edges nor self-loops.

Note that you are not required to minimize the number of nodes or edges. It can be proven that if constructing a graph in which the return value of `BDFS()` is K is possible, then there exists one that satisfies all the constraints above. If there are several solutions, you can output any of them.

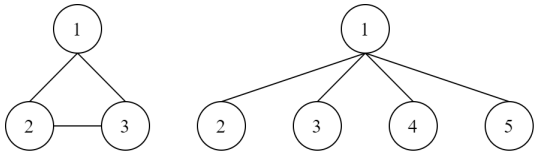
Examples

standard input	standard output
8	3 3 1 2 1 3 2 3
1	-1 -1
23	5 7 4 5 2 3 3 1 2 4 4 3 2 1 1 5

Note

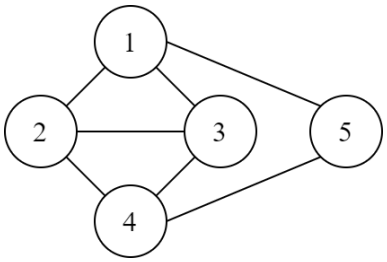
Explanation for the sample input/output #1

The graph on the left describes the output of this sample. The graph on the right describes another valid solution for this sample.



Explanation for the sample input/output #3

The following graph describes the output of this sample.

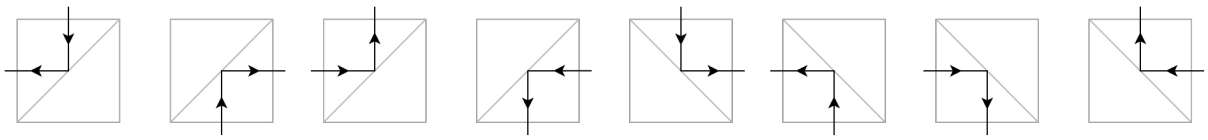


Problem M. Mirror Maze

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	1024 megabytes

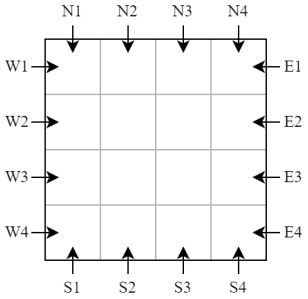
You are given a grid of R rows (numbered from 1 to R from north to south) and C columns (numbered from 1 to C from west to east). Every cell in this grid is a square of the same size. The cell located at row r and column c is denoted as (r, c). Each cell can either be empty or have a mirror in one of the cell's diagonals. Each mirror is represented by a line segment. The mirror is type 1 if it is positioned diagonally from the southwest corner to the northeast corner of the cell, or type 2 for the other diagonal.

These mirrors follow the law of reflection, that is, the angle of reflection equals the angle of incidence. Formally, for type 1 mirror, if a beam of light comes from the north, south, west, or east of the cell, then it will be reflected to the west, east, north, and south of the cell, respectively. Similarly, for type 2 mirror, if a beam of light comes from the north, south, west, or east of the cell, then it will be reflected to the east, west, south, and north of the cell, respectively.



You want to put a laser from outside the grid such that all mirrors are hit by the laser beam. There are $2 \cdot (R + C)$ possible locations to put the laser:

- from the north side of the grid at column c, for $1 \le c \le C$, shooting a laser beam to the south;
- from the south side of the grid at column c, for $1 \le c \le C$, shooting a laser beam to the north;
- from the east side of the grid at row r, for $1 \le r \le R$, shooting a laser beam to the west; and
- from the west side of the grid at row r, for $1 \le r \le R$, shooting a laser beam to the east.



Determine all possible locations for the laser such that all mirrors are hit by the laser beam.

Input

The first line consists of two integers R C ($1 \le R, C \le 200$).

Each of the next R lines consists of a string S_r of length C. The c-th character of string S_r represents cell (r, c). Each character can either be `.` if the cell is empty, `/` if the cell has type 1 mirror, or `\` if the cell has type 2 mirror. There is **at least one mirror** in the grid.

Output

Output a single integer representing the number of possible locations for the laser such that all mirrors are hit by the laser beam. Denote this number as k.

If $k > 0$, then output k space-separated strings representing the location of the laser. Each string consists of a character followed **without any space** by an integer. The character represents the side of the grid, which could be `N`, `S`, `E`, or `W` if you put the laser on the north, south, east, or west side of the grid, respectively. The integer represents the row/column number. You can output the strings in any order.

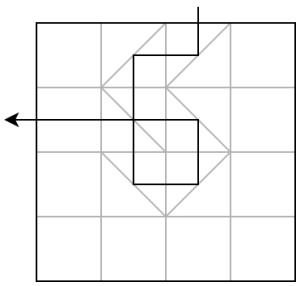
Examples

standard input	standard output
4 4 .//. .\\. .\/.	2 N3 W2
4 6 ./..\. .\...\ ./../\	2 E3 S2
4 4 .... ./\. .\/.	0

Note

Explanation for the sample input/output #1

The following illustration shows one of the solutions of this sample.



Explanation for the sample input/output #2

The following illustration shows one of the solutions of this sample.